

UNIVERSIDAD DE COSTA RICA
ESCUELA DE INGENIERÍA ELÉCTRICA

IE0624
Laboratorio de Microcontroladores

Proyecto Final

Sistema de suministro de agua y comida controlado por IoT

Estudiantes:
Kendall Saborio Picado - B87103
Alexander Rojas Brenes - B86869

Profesor:
Ing. Marco Villalta Fallas

Fecha de entrega: 04/07/2024

Índice

1. INTRODUCCIÓN	2
2. JUSTIFICACIÓN	2
3. OBJETIVOS	3
3.1. Objetivo general	3
3.2. Objetivos específicos	3
4. NOTA TEÓRICA	4
4.1. Placa NODEMCU ESP8266 WIFI	4
4.1.1. Características Eléctricas: ESP8266	5
4.1.2. Diagrama de bloques: ESP8266	5
4.2. Librerías de importancia	5
4.3. IoT: Internet of things	6
4.4. Sensores ultrasónicos (HC-SR04)	6
4.5. Servo motor del kit de Arduino	7
4.6. Bomba de agua	8
4.7. Blynk IoT	9
4.8. Lista de componentes y precios	10
5. DESARROLLO Y ANÁLISIS DE RESULTADOS	11
5.1. Desarrollo de la solución	11
5.1.1. Dispensador de comida/Servo/Stepper	11
5.1.2. Dispensador de agua/Bomba/Relé/Transistor	13
5.1.3. Tazas de agua y comida/Sensores ultrasónicos	14
5.1.4. Circuito general	15
5.1.5. Desarrollo de <i>software</i>	16
5.1.6. Arduino Cloud/Blynk	17
5.2. Análisis de los resultados	18
5.3. Caso A: Tazas Vacías	18
5.4. Caso B: Llenando taza Agua	19
5.5. Caso C: Llenando taza de Comida	21
6. CONCLUSIONES Y RECOMENDACIONES	25
7. Anexos	27
7.1. Hoja del fabricante del (...): información general	27
7.2. Hoja del fabricante del ESP8266: información general	29

1. INTRODUCCIÓN

Este proyecto se centra en el desarrollo de un dispensador de comida y agua para mascotas con capacidades IoT, el proyecto ha sido un esfuerzo integral para crear una solución automatizada y eficiente para el cuidado de las mascotas. Este sistema utiliza sensores ultrasónicos para monitorear los niveles de comida y agua, y permite a los propietarios controlar y supervisar el dispensador a través de una aplicación móvil, utilizando la plataforma Blynk.

Durante el desarrollo del proyecto, se realizaron ajustes significativos en el diseño y la selección de componentes para cumplir con las prioridades definidas. La adaptación a las limitaciones técnicas, como el cambio de motores y la búsqueda de alternativas, permitió avanzar en el desarrollo del dispensador, logrando así el objetivo principal del proyecto.

A lo largo del proceso, se enfrentaron varios problemas técnicos, como la falta de potencia del ESP8266 y dificultades con Arduino Cloud. Para superar estos obstáculos, se implementaron estrategias efectivas, incluyendo la transición a la plataforma Blynk, lo que garantizó el funcionamiento del sistema de dispensación de comida y agua. Esto demostró una notable flexibilidad y capacidad de resolución de problemas.

Se logró implementar funcionalidades básicas de IoT mediante la integración de Blynk, permitiendo el monitoreo remoto del estado de los recipientes y el control del sistema desde un dispositivo móvil. Esta implementación cumplió parcialmente con el objetivo de automatización del suministro, facilitando el control a distancia del sistema.

La construcción del hardware, que incluye el dispensador de comida, el sistema de bombas para agua y los sensores ultrasónicos, fue completada exitosamente. Este aspecto del proyecto proporcionó una base sólida para el funcionamiento básico del sistema, permitiendo que el dispensador cumpliera con sus funciones principales de manera efectiva.

Para consultar el trabajo realizado, puede consultarse el siguiente repositorio de trabajo, en la rama llamada "Proj": https://github.com/SaboEIE/IE-0624-IS-2024_B87103/tree/Proj. El código principal en C (archivo .ino), puede consultarse en la ruta Proj/src.

2. JUSTIFICACIÓN

Este proyecto busca mejorar el bienestar animal al garantizar que las mascotas y animales de granja reciban los nutrientes necesarios en los momentos adecuados. Además, ofrece una solución conveniente para los propietarios, especialmente aquellos con horarios ocupados, al automatizar el proceso de alimentación y liberarlos de la preocupación por mantener horarios específicos de alimentación. El uso de tecnología conectada a Internet permite el monitoreo remoto de la alimentación de los animales, lo que resulta especialmente útil y permite a los propietarios estar ausentes durante largos períodos de tiempo. Además, la automatización del suministro de comida y agua facilita el control preciso del consumo, evitando el desperdicio y garantizando un suministro adecuado sin excesos.

3. OBJETIVOS

3.1. Objetivo general

Diseñar y desarrollar un dispositivo suplidor inteligente de agua y comida para animales utilizando ESP8266, con el fin de mejorar el bienestar animal y proporcionar una solución conveniente para los propietarios, automatizando el proceso de alimentación y permitiendo el monitoreo remoto de la alimentación de los animales.

3.2. Objetivos específicos

1. Implementar un sistema de suministro de agua y comida que permita tanto la solicitud directa del usuario como una rutina automatizada con tiempos predefinidos, utilizando tecnología ESP8266 para controlar los mecanismos de dispensación.
2. Integrar la aplicación Arduino Cloud para enviar notificaciones al teléfono móvil del usuario, informando sobre el contenido de los recipientes y los tanques principales de agua y comida, así como para permitir la programación de suministros automáticos y solicitudes adrede.
3. Garantizar un control preciso del consumo de comida y agua mediante la automatización del suministro, evitando el desperdicio y asegurando un suministro adecuado sin excesos.

4. NOTA TEÓRICA

Antes de iniciar con el desarrollo del presente proyecto, es necesario tener a mano una serie de conceptos importantes, los cuales se explican a continuación:

4.1. Placa NODEMCU ESP8266 WIFI

La placa NodeMCU ESP8266 es una plataforma de desarrollo muy accesible y de bajo costo, basada en el microcontrolador ESP8266. Este microcontrolador de 32 bits utiliza la arquitectura Tensilica Xtensa LX106 y puede operar a una frecuencia de entre 80 MHz y 160 MHz. Una de sus características más destacadas es su capacidad Wi-Fi integrada, lo que permite que la placa se conecte a redes Wi-Fi y funcione tanto como punto de acceso como cliente, lo que la hace ideal para proyectos de Internet de las Cosas (IoT).

La NodeMCU ESP8266 ofrece 17 pines GPIO (General Purpose Input/Output) para conectar sensores, actuadores y otros periféricos (4 pueden configurarse como PWM a 3.3V). También incluye pines para comunicación I2C, SPI, UART, ADC, entre otros. Su programación es muy accesible, ya que puede hacerse usando el entorno de desarrollo Arduino IDE, lo que la hace ideal para principiantes y entusiastas. Además, puede programarse en lenguaje Lua a través del firmware NodeMCU.

El microcontrolador ESP8266 cuenta con una memoria flash interna, de 4 MB, que permite almacenar el firmware y otros datos necesarios para el funcionamiento del dispositivo. Gracias a sus capacidades Wi-Fi y su bajo costo, la NodeMCU ESP8266 es utilizada en una amplia variedad de proyectos, desde la automatización del hogar hasta sistemas de monitoreo remoto y dispositivos IoT.

La placa puede ser alimentada mediante un cable micro-USB o a través de los pines de alimentación (VIN y GND), con un voltaje de funcionamiento de 3.3V. Además, es compatible con plataformas de desarrollo como Blynk, facilitando la creación de aplicaciones móviles para controlar y monitorear dispositivos conectados.

A continuación se puede ver el Diagrama de pines de la placa:

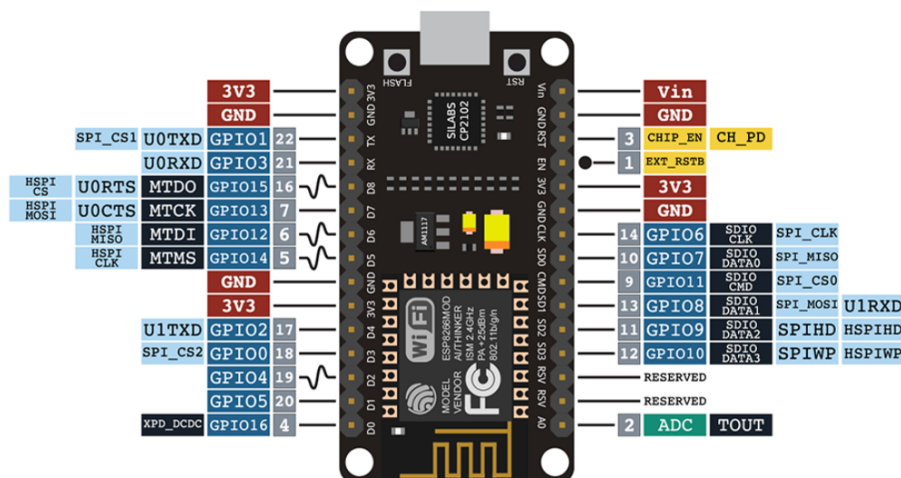


Figura 1: Diagrama de Pines NodeMCU ESP8266. (Fuente: [1])

4.1.1. Características Eléctricas: ESP8266

A continuación se muestran las características electricas del ESP8266 según su hoja del fabricante [2]:

Parameters	Conditions	Min	Typical	Max	Unit
Operating Temperature Range	-	-40	Normal	125	C
Maximum Soldering Temperature	IPC/JEDEC J-STD-020	-	-	260	C
Working Voltage Value	-	2.5	3.3	3.6	V
V_{IL}	-	-0.3	-	$0.25 V_{IO}$	V
V_{IH}	-	$0.75 V_{IO}$	-	3.6	V
V_{OL}	-	-	-	$0.1 V_{IO}$	V
V_{OH}	-	$0.8 V_{IO}$	-	-	V
I_{max}	-	-	-	12	mA
Electrostatic Discharge (HBM)	TAMB = 25 C	-	-	2	KV
Electrostatic Discharge (CDM)	TAMB = 25 C	-	-	0.5	KV

Tabla 1: Características eléctricas NodeMCU ESP8266

4.1.2. Diagrama de bloques: ESP8266

A continuación se muestra el Diagrama de bloques del ESP8266 según su hoja del fabricante [2]:

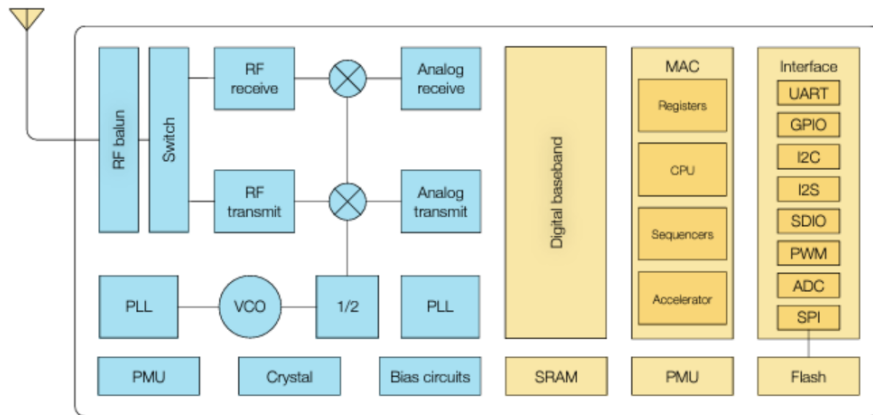


Figura 2: Diagrama de bloques ESP8266. (Fuente: [2])

4.2. Librerías de importancia

A continuación se detallan las librerías que fueron utilizadas en el proyecto:

- [ESP8266WiFi.h](#): permite al ESP8266 conectarse a redes WiFi. Proporciona funciones para escanear redes disponibles, conectar o desconectar de una red, y obtener información sobre la conexión actual, como la intensidad de la señal y la dirección IP asignada.
- [BlynkSimpleEsp8266.h](#): diseñada para integrar el ESP8266 con la plataforma Blynk. Permite conectar el dispositivo a la aplicación móvil Blynk para crear interfaces de usuario con botones, *sliders* y otros *widgets* para controlar el ESP8266 y recibir datos en tiempo real.

- Servo.h: permite controlar servomecanismos con el ESP8266. Proporciona funciones para configurar el ángulo del servo motor, establecer la velocidad de movimiento, y otras operaciones básicas necesarias para controlar servos en proyectos de robótica o automatización.

4.3. IoT: Internet of things

El Internet de las cosas (IoT, por sus siglas en inglés) se refiere a una red abierta y completa de objetos inteligentes que tienen la capacidad de autoorganizarse, compartir información, datos y recursos, reaccionando y actuando ante situaciones y cambios en el entorno. Es un concepto que ha evolucionado desde la idea de que la primera versión de Internet trataba sobre datos creados por personas, mientras que la siguiente versión se trata de datos creados por cosas [3].

4.4. Sensores ultrasónicos (HC-SR04)

El HC-SR04 es un sensor de distancia ultrasónico que se destaca por su bajo costo y facilidad de uso. Este sensor es capaz de medir distancias en un rango de 2 cm a 400 cm, lo que lo convierte en una herramienta popular en proyectos de robótica y automatización para evitar obstáculos y medir distancias [4]. A continuación se detallan los pasos del funcionamiento del sensor:

1. **Emisión de onda ultrasónica:** el sensor está equipado con dos transductores ultrasónicos: uno para enviar las ondas (transmisor) y otro para recibirlas (receptor). Cuando se activa el pin Trig (*Trigger*) enviando un pulso alto de al menos 10 μ s, el sensor emite una serie de ondas ultrasónicas a una frecuencia de 40 kHz.
2. **Reflexión de onda:** las ondas ultrasónicas viajan a través del aire y se reflejan en un objeto situado frente al sensor.
3. **Recepción de onda:** el transductor receptor detecta las ondas reflejadas y genera un pulso en el pin Echo. La duración de este pulso es proporcional al tiempo que tardaron las ondas en viajar al objeto y regresar al sensor.
4. **Cálculo de distancia:** La distancia se calcula utilizando la siguiente fórmula:

$$Distancia(d) = \frac{V \times T}{2}$$

Donde V es la velocidad del sonido en el aire y T es el tiempo medido en microsegundos (μ s) desde el envío hasta la recepción de las ondas.

En la Figura 3, puede consultarse el comportamiento descrito, además de la disposición de pines:

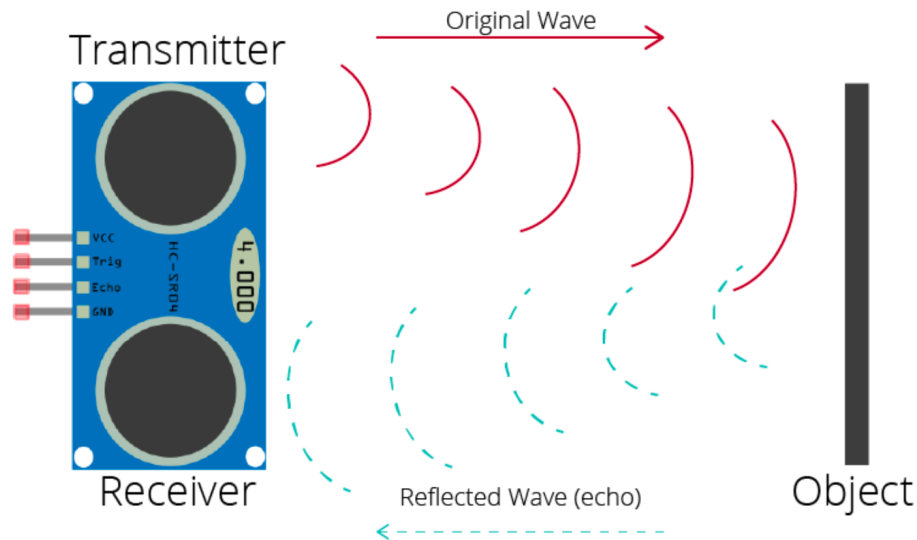


Figura 3: Funcionamiento del sensor ultrasónico HC-SR04 y disposición de pines del dispositivo.
(Fuente: [5])

4.5. Servo motor del kit de Arduino

Según se detalla en [6], un servomotor es un dispositivo de actuador utilizado en diversos campos como la robótica, vehículos RC y modelos aéreos. Se destaca por su capacidad de control preciso de posición, lo que lo hace adecuado para tareas que requieren movimientos exactos y de alta fuerza.

Un servomotor típico está compuesto por un motor eléctrico, un potenciómetro, electrónica de control y una caja de engranajes. El potenciómetro mide la posición actual del eje, y la electrónica de control compara esta posición con la posición deseada, ajustando el motor para mantener la posición del eje en el valor objetivo. Este proceso de ajuste continuo constituye un sistema de control en bucle cerrado. Lo anterior puede apreciarse de manera gráfica en la Figura 4:

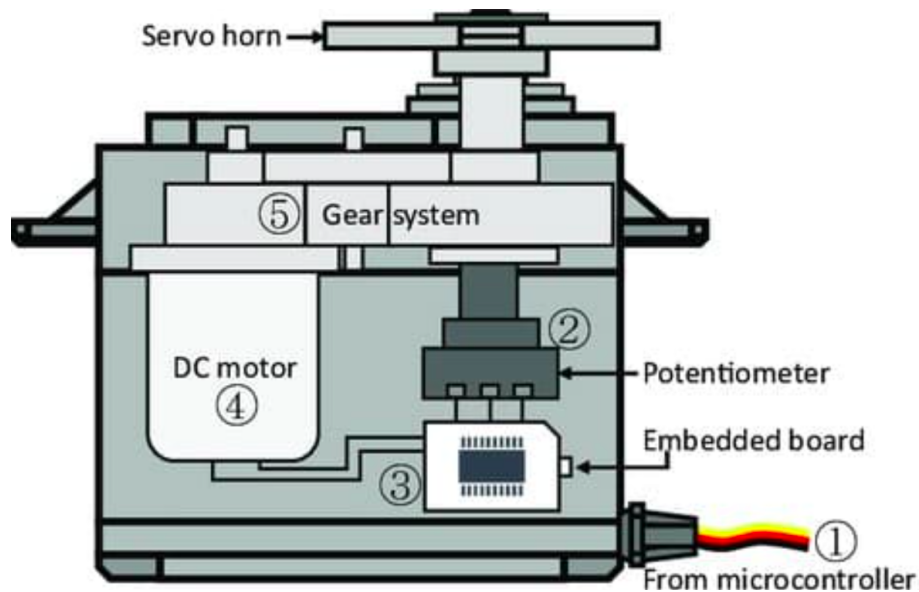


Figura 4: Composición de un servo motor. (Fuente: [6])

Para conectar un servomotor a un microcontrolador, se deben conectar tres cables: alimentación (VCC) a 5V, tierra (GND) a GND, y señal a un pin digital del dispositivo de control. Es fundamental considerar que, si se utilizan múltiples servos o servos de mayor tamaño, se debe utilizar una fuente de alimentación externa para evitar dañar el microcontrolador.

4.6. Bomba de agua

Tal como se describe en [7], este componente cuenta con un motor de 6V y 0.25A de alta resistencia, un cuerpo de plástico termoplástico resistente, y es completamente sumergible, aunque también puede funcionar en seco sin dañarse. La bomba, opera a un rango de voltaje de 2.5 a 6V, ofrece una elevación máxima de 40 a 110 cm y un caudal de 80 a 120 L/h. Sus dimensiones son un diámetro de aproximadamente 24 mm, una longitud de 45 mm y una altura de 33 mm. Equipado con un motor magnético sin escobillas y una vida útil de 500 horas de funcionamiento continuo, este dispositivo es ideal para aplicaciones diversas como fuentes, cascadas, y sistemas de riego automatizados, garantizando un movimiento eficiente y confiable del agua. En la Figura 5, puede observarse el tipo de mini bomba de agua que se utilizará en el proyecto:



Figura 5: Mini-bomba de agua. (Fuente: [7])

Una bomba de agua funciona mediante un impulsor que gira para crear una fuerza centrífuga, lo que genera una baja presión en el centro y succiona el agua hacia el interior de la bomba a través del puerto de succión. El agua es entonces empujada hacia afuera mediante la presión generada y dirigida a través del puerto de descarga gracias al diseño de la voluta. El motor impulsa el impulsor, y en algunos casos, la bomba necesita ser primada para eliminar el aire y asegurar un flujo continuo de agua [8].

4.7. Blynk IoT

Tal como se explica en [9], Blynk es una plataforma en la nube para IoT de bajo código que simplifica la creación, implementación y gestión de aplicaciones IoT. Permite construir aplicaciones móviles y web personalizables sin necesidad de programación, gestionar dispositivos, datos y actualizaciones, y garantizar seguridad empresarial en todo el ciclo de vida del IoT. Blynk puede ser conectado con varios dispositivos como SP32, ESP8266, Arduino Uno, Raspberry Pi, Particle Photon, Seeed Wio Terminal y TI CC3220 y ofrece diversos métodos para conectar dispositivos a su plataforma en la nube:

- **Blynk Library:** Una biblioteca C++ que proporciona una conexión bidireccional de baja latencia entre el dispositivo y la nube de Blynk, manteniendo el dispositivo siempre conectado.
- **Blynk.Edgent:** Una solución empaquetada que incluye Blynk.Inject para provisión dinámica de credenciales y Blynk.Air para actualizaciones de firmware OTA, ideal para hardware compatible con esta opción.
- **Blynk.NCP:** Una pila de software para módulos de procesamiento de red (NCP) que maneja la conectividad (WiFi, Ethernet, Celular), permitiendo al MCU principal usar una biblioteca cliente ligera para la comunicación.
- **HTTP(s) API:** Un protocolo estándar que permite a los dispositivos conectarse periódicamente a la nube para transferir datos, ideal para dispositivos celulares que envían datos en lotes.
- **MQTT API:** Un protocolo de comunicación bidireccional seguro que admite autenticación, actualizaciones de datos, eventos y más, ideal para proyectos que utilizan bibliotecas MQTT o plataformas similares.

Estos métodos ofrecen flexibilidad en el desarrollo y gestión de aplicaciones IoT, adaptándose a diferentes necesidades y tipos de hardware [9].

4.8. Lista de componentes y precios

En la Tabla 2 pueden consultarse los componentes utilizados y sus precios tomando como referencia los precios del sitio web de la tienda de componentes MicroJPM (<https://www.microjpm.com/>):

Tabla 2: Lista de componentes.

Componente	Cantidad	Precio
NodeMCU ESP8266	1	\$10,95
Batería de 9 V	1	\$5.64
Servo motor	1	\$5,95
Mini bomba agua sumergible	1	\$5,50
Resistor de 2 k Ω	2	\$0,80
Transistor 2N2222	1	\$0,25
Relé	1	\$1.08
Diodo	1	\$0.095
Jumpers	60	\$6.90
Materiales de construcción	-	\$11.71
Mano de obra	-	\$19.99
Total	-	\$68.86

Según la Tabla 2, para realizar este proyecto son necesarios ₡36176.15, según el valor del dólar al momento de escribir el presente informe.

5. DESARROLLO Y ANÁLISIS DE RESULTADOS

A continuación se muestran los métodos de desarrollo utilizados para resolver cada uno de los problemas planteados, así como la funciones utilizadas, el *hardware* construido y los resultados obtenidos:

5.1. Desarrollo de la solución

En esta sección, se explica el proceso de desarrollo de *hardware* y *software* utilizado para diseñar e implementar la solución planteada para el proyecto:

5.1.1. Dispensador de comida/Servo/Stepper

Inicialmente se realizó una búsqueda de posibles diseños existentes para tomar ideas y construir el dispensador de comida.



Figura 6: Primer diseño consultado. (Fuente: [10])

Este primer diseño es muy interesante, note que hace uso de un stepper motor así como de una especie de varille que controla la tapa de apertura del contenedor. Inicialmente se implementó uno diseño prácticamente idéntico, sin embargo un aspecto importante a considerar era que en el escenario de este proyecto, se desea que el reservorio principal tenga alimento para más de un refill. Esto generaba que la cantidad e alimento fuera considerable y que su peso cayera sobre la integridad de la tapa. Por que la tapa al intentar abrir, necesitaba el torque necesario para levantar el alimento. Al seguir consultando diseños propuestos, se encontró el siguiente:

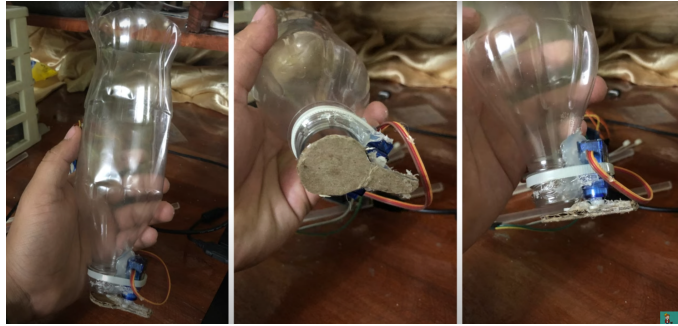


Figura 7: Segundo diseño consultado. (Fuente: [11])

De este diseño lo que resultó llamativo fué el uso del servomotor. Por lo qué de aca surgió la idea de migrar al uso de servo y crear una especie de portón corredizo que permitiera el flujo de comida. Para este portón se utilizó una malla reciclada, que normalmente se encuentra en la base de los sartenes de cocina. El diseño propuesto de dispensador de alimento para este trabajo el siguiente:



Figura 8: Diseño propuesto.

Note que el diseño propuesto es una combinación de las ideas consultadas, más mejoras como por ejemplo el aumento de capacidad de almacenamiento pegando más tubo al codo. Se añadió una placa metálica plateada de seguridad, que mantiene la tapa del reservorio presionada, evitando así cualquier desperdicio o fuga de alimento. Note que se está usando el servomotor, es importante mencionar que inicialmente se realizaron pruebas de funcionamiento usando Arduino con sus pines de 5V. Sin embargo una vez se consiguió el microcontrolador ESP8266 que es el microcontrolador seleccionado para este proyecto, este solo cuenta con pines 3.3v y menos corriente. Por lo que se percibió una disminución en el rendimiento y velocidad de operación del servomotor.

5.1.2. Dispensador de agua/Bomba/Relé/Transistor

Para lo que es el reservorio de agua, se decidió utilizar una botella plástica esto por disponibilidad del material, su naturaleza resistente a los líquidos y su capacidad de almacenaje. En ambos reservorios tanto de comida como de agua, se añadió un soporte, que permite apoyar los reservorios a cualquier tipo de silla o mueble.



Figura 9: Imagen del Reservorio de agua con soporte.

Se obtuvo una bomba de agua del tipo sumergible, que no cuenta con alguna patilla de control. Por lo que se vio la necesidad de proponer un circuito de control con relé. Inicialmente antes de tener el ESP8266, se realizaron simulaciones y pruebas con el pin de alimentación de arduino UNO de 3.3 V, en el que se lograba activar el relé. Sin embargo al momento de migrar al ESP8266, el relé no se activaba a pesar de tener la misma tensión. En este momento al consultar la literatura nos damos cuenta que los pines GPIO del ESP8266 tienen una corriente de salida de hasta 12 mA,

mientras que los de Arduino de hasta 40 mA en este momento se percibe que es un problema por baja corriente. Investigando se encontró el circuito propuesto en [12], el cual resuelve este problema:

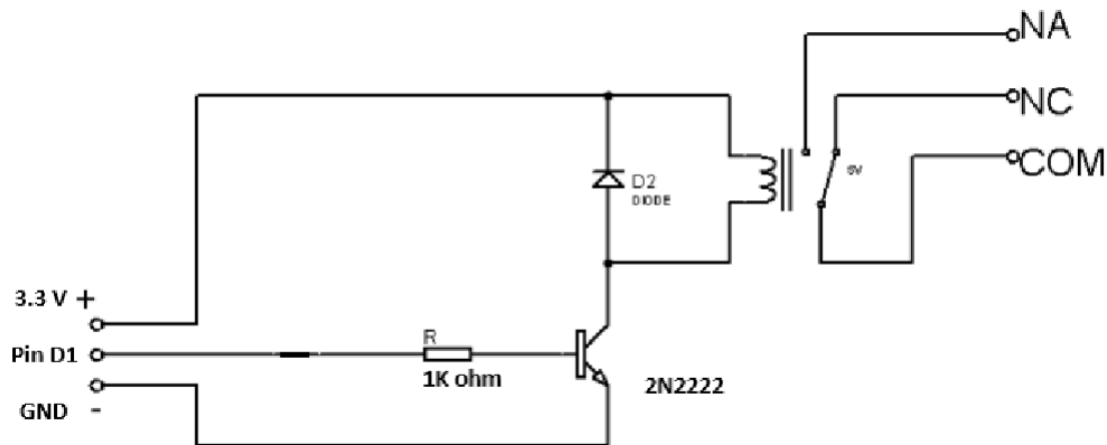


Figura 10: Imagen del Reservorio de agua con soporte.

El uso de un transistor 2N2222 permite tener control sobre el relé usando una patilla GPIO del ESP8266 en la base del transistor. Así como elevar la corriente debido a la ganancia que poseen los transistores. El diodo se utiliza como protección a los picos generados en las transiciones del relé. De esta manera se logró realizar el control de la bomba con ESP8266.

Un problema interesante que surgió en el bombeo entre el reservorio principal y la taza de agua del animal fué un fenómeno denominado como sifón. Un sifón es una diferencia de presión que permite que el agua fluya de un nivel más alto a uno más bajo, incluso sin la ayuda de la bomba.[13] Este fenómeno hacía que el agua fluyera hacia la taza del animal aun cuando la bomba se apagaba. La solución planteada fué variar la altura del reservorio principal.

5.1.3. Tazas de agua y comida/Sensores ultrasónicos

Para realizar las mediciones de distancia en las tazas de comida y agua, fue necesario colocar los sensores ultrasónicos en la parte superior de estas. Para fijar el sensor y evitar movimientos verticales y horizontales que insertaran ruido a las mediciones de nivel, se utilizó porcelana fría, la cual representa un buen material en la medida en que seca bastante rápido y evita cualquier movimiento. En la Figura 11, puede verse la forma en la que los sensores fueron colocados en cada taza:



Figura 11: Taza de agua con el sensor ultrasónico adherido a este utilizando porcelana fría.

5.1.4. Circuito general

Una vez que se tenían todas las partes completadas, solamente restaba unir todas estas utilizando el microcontrolador elegido para el proyecto. En la Figura 12 puede consultarse de manera general y a alto nivel el circuito construido:

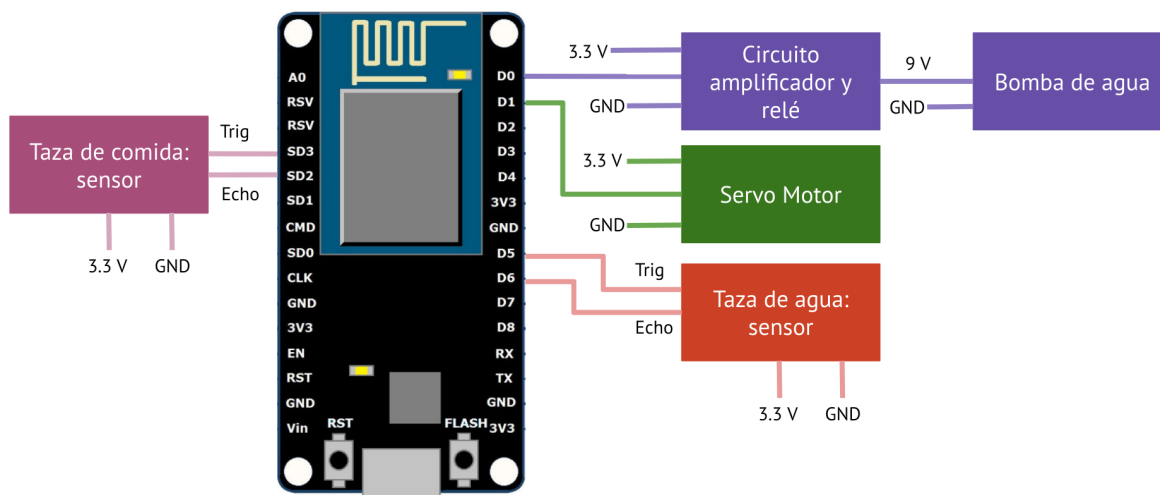


Figura 12: Disposición general del circuito utilizado para el proyecto. (Elaboración propia).

Como puede verse en la Figura 12, el circuito está dividido en tres etapas:

1. **Tazas de agua y de comida:** cada una cuenta con un sensor ultrasónico que se encarga de la medición de proximidad del alimento y agua, resultando en un monitoreo constante del nivel en ambos casos.

2. **Servo motor:** este motor es controlador por el ESP8266, permite la distribución de comida en la taza de comida, cuando sea necesario. El motor tiene el objetivo de abrir el portón del contenedor principal de comida.
3. **Bomba de agua:** esta sección cuenta con dos partes. En primer lugar se tiene el circuito amplificador de corriente para poder accionar el relé de activación de la bomba de agua, este es alimentado por el ESP8266. En segundo lugar, se tiene la bomba de agua como tal, la cual se encargar de la distribución de agua desde el contenedor principal a la taza de agua cuando sea necesario.

A continuación se muestra el circuito completo conectado en su etapa de pruebas:

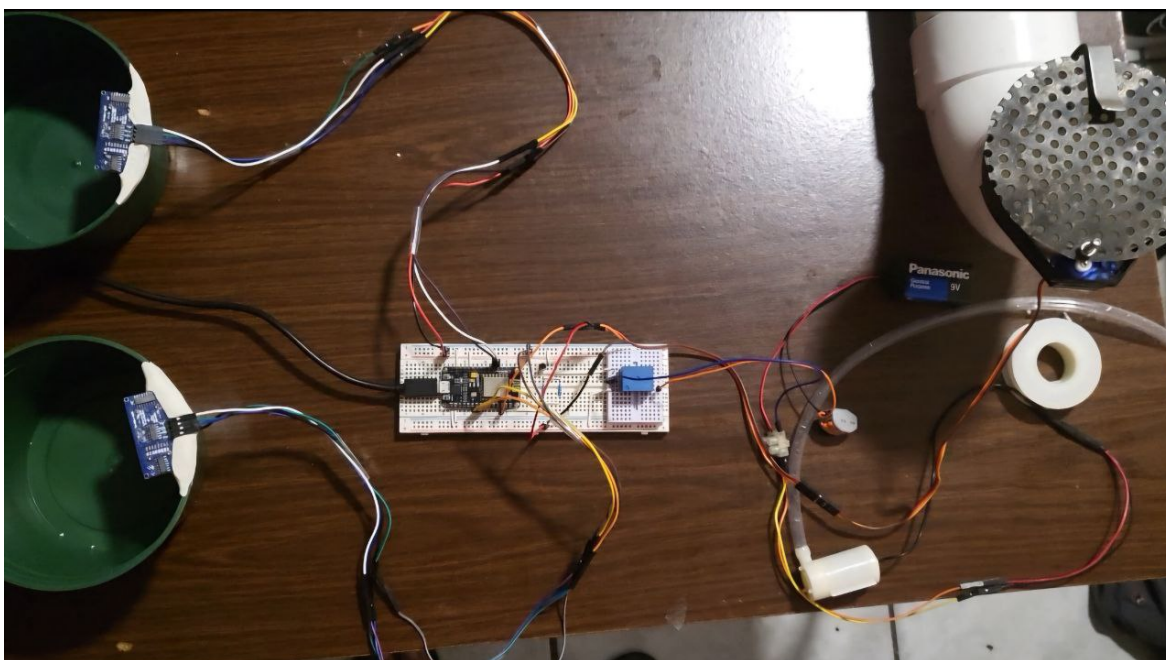


Figura 13: Imagen del circuito completo en su etapa de pruebas. (Elaboración propia).

5.1.5. Desarrollo de *software*

A continuación podrá consultarse una descripción de alto nivel de cada una de las funciones implementadas en el código .ino:

- `setup()`: esta corresponde a la función principal de configuración, en esta se configuran las siguientes partes:
 - `setup_servo()`: se encarga de la configuración básica del servo motor.
 - `setup_pump()`: se encarga de la configuración básica de los pines de control utilizados para controlar el circuito de amplificación que a su vez se encarga de controlar la bomba de agua.
 - `setup_ultrasonic()`: configura los pines de *trigger* y *echo* de los dos sensores ultrasónicos utilizados.

- `distance_measure(int trigPin, int echoPin)`: esta función envía un pulso de 10 microsegundos al pin de disparo del sensor ultrasónico (`trigPin`), y luego mide el tiempo que tarda en recibir el pulso reflejado en el pin de eco (`echoPin`). Utiliza esta duración para calcular y retornar la distancia en centímetros.
- `water_level_bowl(int trigPin, int echoPin)`: mide la distancia desde el sensor ultrasónico ubicado en la taza de agua. Si la distancia medida es mayor o igual a 7 cm, se considera que la taza está vacía y se muestra un mensaje en el serial. Si se recibe una señal de Blynk para llenar la taza, la bomba de agua se activa hasta que la distancia medida sea menor a 4 cm. Durante este proceso, se envían los valores de nivel de agua a la aplicación Blynk.
- `food_level_bowl(int trigPin, int echoPin)`: mide la distancia desde el sensor ultrasónico ubicado en la taza de comida. Si la distancia medida es mayor o igual a 7 cm, se considera que la taza está vacía y se muestra un mensaje en el serial. Si se recibe una señal de Blynk para llenar la taza, el servo motor se mueve a 180 grados para dispensar comida hasta que la distancia medida sea menor a 5.5 cm. Durante este proceso, se envían los valores de nivel de comida a la aplicación Blynk.
- `status_checker()`: verifica los niveles en las tazas de comida y agua.
- `loop()`: Mantiene la conexión con el servidor de Blynk mediante `Blynk.run()`, gestionando la comunicación con la aplicación móvil. Además, llama a `status_checker()` para monitorear y controlar los niveles de comida y agua en los recipientes y contenedores, tomando acciones en consecuencia.

5.1.6. Arduino Cloud/Blynk

Tal como había sido planteado al inicio del proyecto, se quería utilizar la plataforma Arduino Cloud para manejar el sistema construido utilizando IoT. A pesar de que se configuró e intentó la implementación de esta, se presentaron varios problemas:

- Errores al intentar conectar el ESP8266 a la plataforma. A pesar de que el agente necesario fue instalado, la plataforma no era capaz de identificar el dispositivo todo el tiempo, lo que causaba inestabilidad al momento de hacer *upload*.
- En las ocasiones en las que el dispositivo sí pudo conectarse a la plataforma, se tuvieron problemas con la conexión WiFi. El error correspondía a un problema con el *broker* de MQTT, ya que no podía acceder al puerto dispuesto para la comunicación, independientemente de la red en la que el dispositivo se conectara.

Es por lo anterior que se decidió utilizar cambiar de plataforma y utilizar Blynk para realizar las mismas tareas planteadas para Arduino Cloud. La conexión entre la plataforma Blynk y el ESP8266 se logra mediante la función `Blynk.begin(auth, ssid, pass)`, que establece la conexión con el servidor de Blynk utilizando credenciales definidas. Dentro del bucle principal `loop()`, se llama continuamente a `Blynk.run()` para mantener activa la conexión y permitir la comunicación bidireccional. Los comandos y datos se intercambian a través de funciones como `BLYNK_WRITE(V0)` y `BLYNK_WRITE(V1)`, que responden a los cambios en los widgets de la aplicación, permitiendo controlar remotamente el nivel de comida y agua en las tazas.

5.2. Análisis de los resultados

5.3. Caso A: Tazas Vacías

A continuación se analiza el escenario en el que ambas tazas se encuentran vacías.



Figura 14: Imagen del dispensador con las tazas vacías.

En este escenario se espera que en terminal se realice la impresión que indique que tanto la taza de agua como la de comida se encuentran vacías. A continuación se muestra como efectivamente se cumple lo anterior:

```
20:00:43.234 -> Taza de agua vacía. Presione Water para llenar.  
20:00:44.253 -> Taza de comida vacía. Presione food para llenar.  
20:00:45.253 -> Taza de agua vacía. Presione Water para llenar.  
20:00:46.252 -> Taza de comida vacía. Presione food para llenar.  
20:00:47.253 -> Taza de agua vacía. Presione Water para llenar.  
20:00:48.235 -> Taza de comida vacía. Presione food para llenar.  
20:00:49.234 -> Taza de agua vacía. Presione Water para llenar.  
20:00:50.266 -> Taza de comida vacía. Presione food para llenar.  
20:00:51.261 -> Taza de agua vacía. Presione Water para llenar.
```

Figura 15: Impresión en Serial monitor con las tazas vacías.

Además se obtiene en el *dashboard* de Blynk, que los niveles de las tazas reflejen valores bajos de acuerdo a su condición de vacíos:

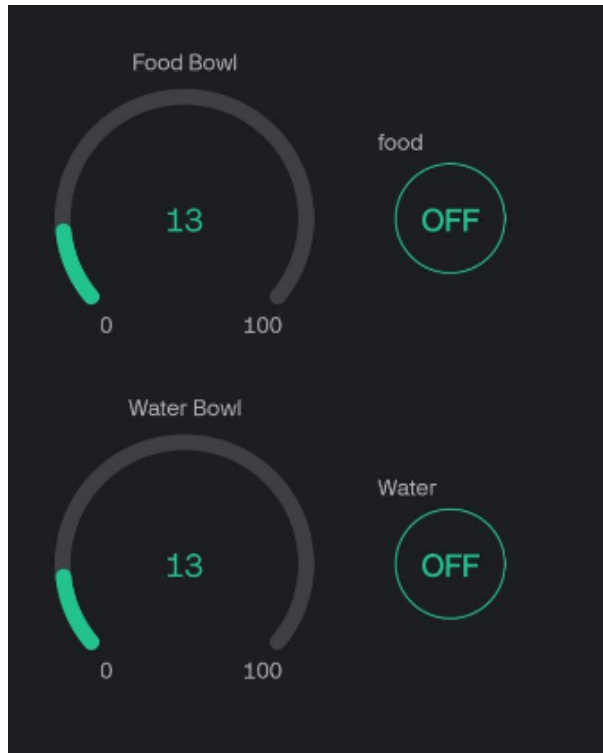


Figura 16: *Dashboard* de Blynk con las tazas vacías.

De este escenario A se puede analizar que se están imprimiendo lecturas de vacío esperadas. Esto indica que los sensores ultrasónicos están detectando adecuadamente cuando las tazas están vacías. El *dashboard* muestra valores que concuerdan con lo que esta pasando en las tazas, por lo que la comunicación entre los sensores y el *dashboard* funciona como se espera.

5.4. Caso B: Llenando taza Agua

A continuación se analiza el escenario en el se acciona el botón de llenar taza de agua.



Figura 17: Imagen de taza de agua con el llenado en progreso.

Mientras se esta realizando el llenado de la taza de agua, en terminal se obtiene la siguiente impresión:

```
20:02:45.392 -> Llenando taza de agua.  
20:02:46.432 -> Llenando taza de agua.  
20:02:47.387 -> Llenando taza de agua.  
20:02:48.395 -> Llenando taza de agua.  
20:02:49.390 -> Llenando taza de agua.  
20:02:50.389 -> Llenando taza de agua.  
20:02:51.393 -> Llenando taza de agua.  
20:02:52.396 -> Llenando taza de agua.  
20:02:53.407 -> Llenando taza de agua.  
20:02:54.395 -> Llenando taza de agua.  
20:02:55.407 -> Llenando taza de agua.
```

Figura 18: Impresión con Serial monitor con llenado en progreso de taza de agua.

A continuación una foto del *dashboard* de Blynk con el llenado en progreso:

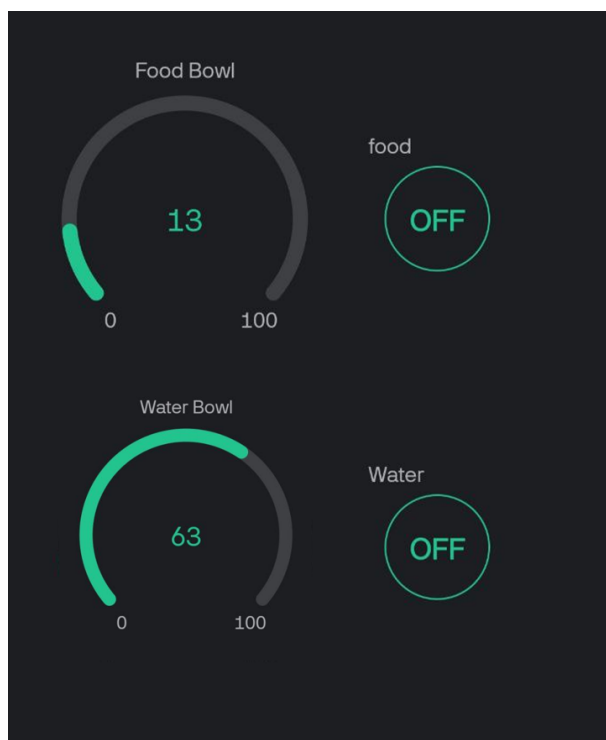


Figura 19: *Dashboard* con llenado en progreso de taza de agua.

Al finalizar el llenado, se imprime lo siguiente en terminal:

```
20:04:21.299 -> Taza de agua llena.  
20:04:22.272 -> Taza de agua llena.  
20:04:23.259 -> Taza de agua llena.  
20:04:24.262 -> Taza de agua llena.  
20:04:25.262 -> Taza de agua llena.  
20:04:26.306 -> Taza de agua llena.
```

Figura 20: Impresión con Serial monitor con taza de agua llena.

De este escenario, se observa un funcionamiento fluido y coherente entre los diferentes componentes del sistema. Al iniciar la acción desde Blynk, se verifica en el serial monitor que las impresiones de agua se realizan según lo esperado, confirmando que la bomba está operando correctamente. Además, en el *dashboard* de Blynk se registra un progreso continuo y preciso en las mediciones de los niveles de agua, reflejando de manera efectiva el aumento en la cantidad de agua en la taza. Este escenario resalta la integración efectiva entre el control remoto mediante Blynk, la respuesta adecuada de los sensores y la visualización en tiempo real de los datos.

5.5. Caso C: Llenando taza de Comida

A continuación se analiza el escenario en el se acciona el botón de llenar taza de comida

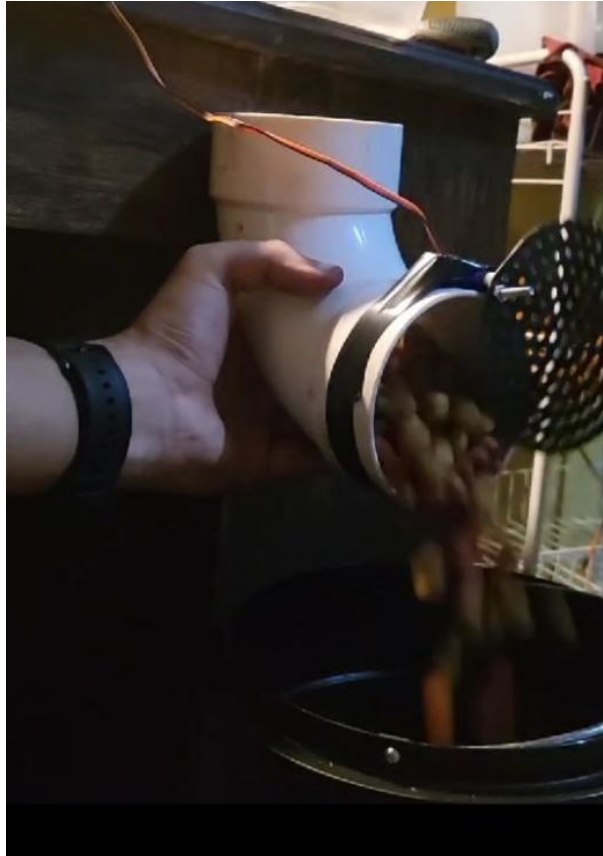


Figura 21: Imagen de Dispensador con llenado en progreso.

Mientras se esta realizando el llenado de la taza de comida, en terminal se obtiene la siguiente impresión:

```
20:08:15.994 -> Llenando taza de comida.  
20:08:17.018 -> Llenando taza de comida.  
20:08:18.013 -> Llenando taza de comida.  
20:08:19.024 -> Llenando taza de comida.  
20:08:20.024 -> Llenando taza de comida.  
20:08:21.019 -> Llenando taza de comida.  
20:08:21.998 -> Llenando taza de comida.  
20:08:23.002 -> Llenando taza de comida.  
20:08:24.013 -> Llenando taza de comida.
```

Figura 22: Impresión con Serial monitor con llenado en progreso de taza de comida.

A continuación una foto del *dashboard* de Blynk con el llenado en progreso:

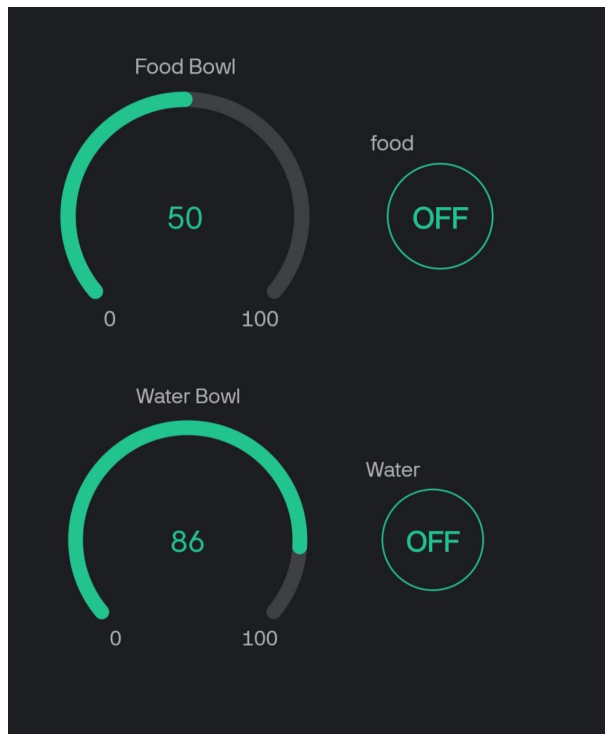


Figura 23: *Dashboard* con llenado en progreso de taza de comida.

Al finalizar el llenado, se imprime lo siguiente en terminal:

```
20:11:09.838 -> Taza de comida llena.  
20:11:10.801 -> Taza de comida llena.  
20:11:11.821 -> Taza de comida llena.  
20:11:12.845 -> Taza de comida llena.  
20:11:13.811 -> Taza de comida llena.  
20:11:14.827 -> Taza de comida llena.  
20:11:15.810 -> Taza de comida llena.
```

Figura 24: Impresión con Serial monitor con taza de comida llena.

A continuación se muestra una imagen de ambas tazas llenas:



Figura 25: Imagen del dispensador con tazas llenas.

El *dashboard* en este punto se muestra de la siguiente manera:

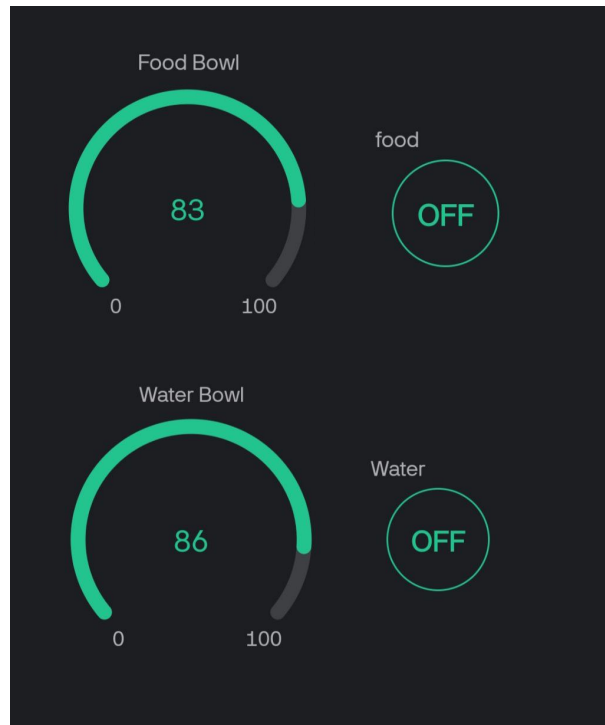


Figura 26: *Dashboard* con tazas llenas.

De este ultimo escenario, se observa un desempeño robusto y coordinado del sistema. Al activar el servo motor a través de Blynk, el monitor serial proporciona retroalimentación en tiempo real con impresiones que indican el progreso del llenado, confirmando la operación adecuada del servo motor y la distribución de comida. El dashboard de Blynk refleja de manera precisa y progresiva el aumento en el nivel de comida en la taza, mostrando una sincronización efectiva entre la acción iniciada y la visualización de datos.

6. CONCLUSIONES Y RECOMENDACIONES

A lo largo del desarrollo del informe se pusieron a prueba una serie de conceptos y prácticas importantes para el manejo básico del ESP8266, para la construcción del sistemas de suministro de comida y agua para animales, utilizando IoT. A continuación algunas conclusiones y recomendaciones recolectadas a lo largo de trabajo realizado:

- A lo largo del proyecto, se realizaron ajustes importantes en el diseño y selección de componentes para cumplir con las prioridades definidas. La adaptación a las limitaciones técnicas, como el cambio de motores y la búsqueda de alternativas, permitió avanzar en el desarrollo del dispensador de comida y agua, logrando el objetivo principal del proyecto.
- El proyecto implicó enfrentar varios problemas técnicos, como la falta de potencia del ESP8266 y dificultades con Arduino Cloud. Se aplicaron estrategias efectivas, como la transición a Blynk, para superar estos obstáculos y garantizar el funcionamiento del sistema de dispensación de comida y agua, demostrando flexibilidad y capacidad de resolución.
- Se logró implementar funcionalidades básicas de IoT mediante la integración de Blynk, lo que permitió el monitoreo remoto del estado de los recipientes y el control del sistema desde un dispositivo móvil. Esta implementación cumplió parcialmente con el objetivo de automatización del suministro y facilitó el control a distancia del sistema.
- La construcción del hardware, que incluye el dispensador de comida, el sistema de bombas para agua y los sensores ultrasónicos, fue completada exitosamente. Este aspecto del proyecto proporcionó una base sólida para el funcionamiento básico del sistema, permitiendo que el dispensador cumpliera con sus funciones principales de manera efectiva.
- Para mejorar el rendimiento del servomotor y manejar mayores cargas, se recomienda replicar el circuito de relé. Este ajuste permitirá al servomotor operar con mayor potencia, lo cual es esencial para la dispensación efectiva de comida en el dispensador y para evitar problemas de desempeño.
- Se sugiere adaptar el diseño del dispensador en función del tipo de animal al que va dirigido. Ajustar el tamaño de los recipientes y el mecanismo de dispensación según las necesidades específicas del animal puede mejorar la eficacia del sistema y proporcionar una solución más adecuada para cada tipo de mascota.

REFERENCIAS BIBLIOGRÁFICAS

- [1] Acrobotic, “Pinout placa nodemcu esp8266,” 2016.
- [2] E. Systems, “Esp8266ex datasheet,” 2023.
- [3] S. Madakam, R. Ramaswamy, and S. Tripathi, “Internet of things (iot): A literature review,” *Journal of Computer and Communications*, vol. 3, no. 5, pp. 164–173, 2015.
- [4] Makerguides, “How to use an HC-SR04 Ultrasonic Distance Sensor with Arduino.” <https://www.makerguides.com/hc-sr04-arduino-tutorial/>, Jan. 2019.
- [5] RNT, “Complete Guide for Ultrasonic Sensor HC-SR04 with Arduino .” <https://randomnerdtutorials.com/complete-guide-for-ultrasonic-sensor-hc-sr04/>, s.f.
- [6] Makerguides, “How to control servo motors with Arduino.” <https://www.makerguides.com/servo-arduino-tutorial/>, Aug. 2020.
- [7] MicroJPM, “Mini bomba de agua sumergible dc 3-6V.” <https://www.microjpm.com/products/mini-bomba-de-agua-sumergible-dc-3-6v/>, 2023.
- [8] Storables, “How Does Water Pump Work.” <https://storables.com/articles/how-does-water-pump-work/>, Jan. 2023.
- [9] Blynk, “Low-code IoT cloud platform with user experience at its core.” <https://blynk.io/>, s.f.
- [10] E. T. De TD, “Alimentador automático bluetooth para mascotas con arduino,” Mar. 2019.
- [11] SalvaTronic, “Dispensador de comida para mascotas con arduino,” Jan. 2024.
- [12] C. Sergio, “Relevador con arduino,” Aug. 2020.
- [13] M. Palomino Ramírez, “Análisis y diseño de sifón,” 2017.

7. Anexos

7.1. Hoja del fabricante del (...): información general

SONGLE RELAY

	RELAY ISO9002	SRD
---	---------------	------------



1. MAIN FEATURES

- Switching capacity available by 10A in spite of small size design for highdensity P.C. board mounting technique.
- UL,CUL,TUV recognized.
- Selection of plastic material for high temperature and better chemical solution performance.
- Sealed types available.
- Simple relay magnetic circuit to meet low cost of mass production.

2. APPLICATIONS

- Domestic appliance, office machine, audio, equipment, automobile, etc.
(Remote control TV receiver, monitor display, audio equipment high rushing current use application.)

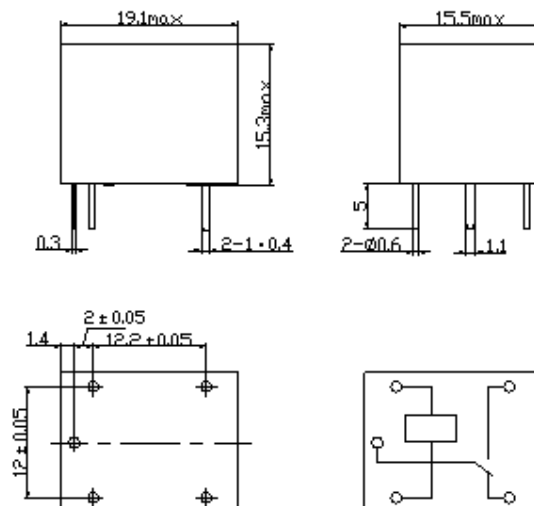
3. ORDERING INFORMATION

SRD	XX VDC	S	L	C
Model of relay	Nominal coil voltage	Structure	Coil sensitivity	Contact form
SRD	03、05、06、09、12、24、48VDC	S:Sealed type	L:0.36W	A:1 form A
		F:Flux free type	D:0.45W	B:1 form B
				C:1 form C

4. RATING

CCC	FILE NUMBER:CH0052885-2000	7A/240VDC
CCC	FILE NUMBER:CH0036746-99	10A/250VDC
UL /CUL	FILE NUMBER: E167996	10A/125VAC 28VDC
TUV	FILE NUMBER: R9933789	10A/240VAC 28VDC

5. DIMENSION(unit:mm) DRILLING(unit:mm) WIRING DIAGRAM



7.2. Hoja del fabricante del ESP8266: información general

ESP8266EX

Datasheet

Note:

It is recommended to use the upgraded model: ESP8684



Version 7.0
Espressif Systems
Copyright © 2023



1.

Overview

Espressif's ESP8266EX delivers highly integrated Wi-Fi SoC solution to meet users' continuous demands for efficient power usage, compact design and reliable performance in the Internet of Things industry.

With the complete and self-contained Wi-Fi networking capabilities, ESP8266EX can perform either as a standalone application or as the slave to a host MCU. When ESP8266EX hosts the application, it promptly boots up from the flash. The integrated high-speed cache helps to increase the system performance and optimize the system memory. Also, ESP8266EX can be applied to any microcontroller design as a Wi-Fi adaptor through SPI/SDIO or UART interfaces.

ESP8266EX integrates antenna switches, RF balun, power amplifier, low noise receive amplifier, filters and power management modules. The compact design minimizes the PCB size and requires minimal external circuitries.

Besides the Wi-Fi functionalities, ESP8266EX also integrates an enhanced version of Tensilica's L106 Diamond series 32-bit processor and on-chip SRAM. It can be interfaced with external sensors and other devices through the GPIOs. Software Development Kit (SDK) provides sample codes for various applications.

Espressif Systems' Smart Connectivity Platform (ESCP) enables sophisticated features including:

- Fast switch between sleep and wakeup mode for energy-efficient purpose;
- Adaptive radio biasing for low-power operation
- Advance signal processing
- Spur cancellation and RF co-existence mechanisms for common cellular, Bluetooth, DDR, LVDS, LCD interference mitigation

1.1. Wi-Fi Key Features

- 802.11 b/g/n support
- 802.11 n support (2.4 GHz), up to 72.2 Mbps
- Defragmentation
- 2 x virtual Wi-Fi interface
- Automatic beacon monitoring (hardware TSF)
- Support Infrastructure BSS Station mode/SoftAP mode/Promiscuous mode



1.2. Specifications

Table 1-1. Specifications

Categories	Items	Parameters
Wi-Fi	Certification	Wi-Fi Alliance
	Protocols	802.11 b/g/n (HT20)
	Frequency Range	2.4 GHz ~ 2.5 GHz (2400 MHz ~ 2483.5 MHz)
	TX Power	802.11 b: +20 dBm
		802.11 g: +17 dBm
		802.11 n: +14 dBm
	Rx Sensitivity	802.11 b: -91 dbm (11 Mbps)
		802.11 g: -75 dbm (54 Mbps)
		802.11 n: -72 dbm (MCS7)
	Antenna	PCB Trace, External, IPEX Connector, Ceramic Chip
Hardware	CPU	Tensilica L106 32-bit processor
	Peripheral Interface	UART/SDIO/SPI/I2C/I2S/IR Remote Control
		GPIO/ADC/PWM/LED Light & Button
	Operating Voltage	2.5 V ~ 3.6 V
	Operating Current	Average value: 80 mA
	Operating Temperature Range	-40 °C ~ 125 °C
	Package Size	QFN32-pin (5 mm x 5 mm)
Software	External Interface	-
	Wi-Fi Mode	Station/SoftAP/SoftAP+Station
	Security	WPA/WPA2
	Encryption	WEP/TKIP/AES
	Firmware Upgrade	UART Download / OTA (via network)
	Software Development	Supports Cloud Server Development / Firmware and SDK for fast on-chip programming
	Network Protocols	IPv4, TCP/UDP/HTTP
	User Configuration	AT Instruction Set, Cloud Server, Android/iOS App

Note:

The TX power can be configured based on the actual user scenarios.



1.3. Applications

- Home appliances
- Home automation
- Smart plugs and lights
- Industrial wireless control
- Baby monitors
- IP cameras
- Sensor networks
- Wearable electronics
- Wi-Fi location-aware devices
- Security ID tags
- Wi-Fi position system beacons



2. Pin Definitions

Figure 2-1 shows the pin layout for 32-pin QFN package.

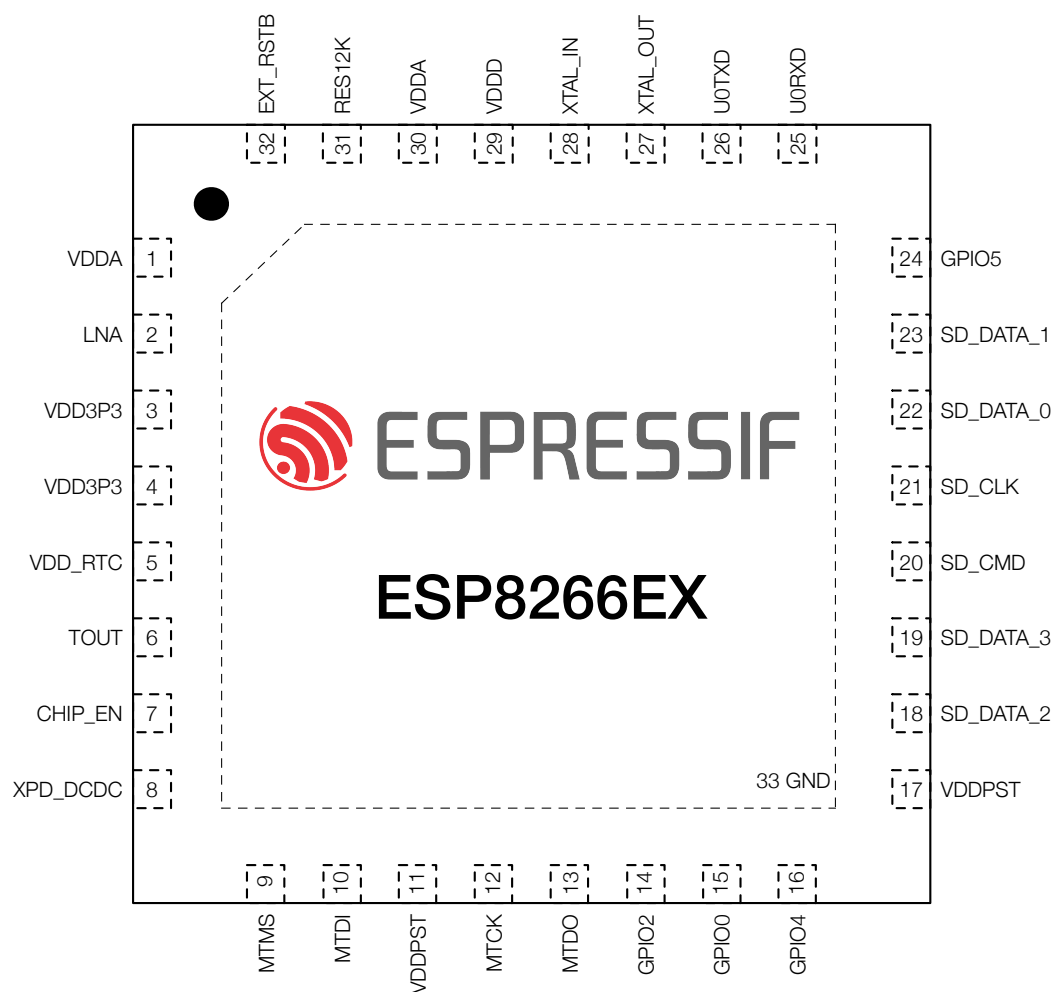


Figure 2-1. Pin Layout (Top View)

Table 2-1 lists the definitions and functions of each pin.

Table 2-1. ESP8266EX Pin Definitions

Pin	Name	Type	Function
1	VDDA	P	Analog Power 2.5 V ~ 3.6 V
2	LNA	I/O	RF antenna interface Chip output impedance = $39 + j6 \Omega$. It is suggested to retain the π -type matching network to match the antenna.
3	VDD3P3	P	Amplifier Power 2.5 V ~ 3.6 V



Pin	Name	Type	Function
4	VDD3P3	P	Amplifier Power 2.5 V ~ 3.6 V
5	VDD_RTC	P	NC (1.1 V)
6	TOUT	I	ADC pin. It can be used to test the power-supply voltage of VDD3P3 (Pin3 and Pin4) and the input power voltage of TOUT (Pin 6). However, these two functions cannot be used simultaneously.
7	CHIP_EN	I	Chip Enable High: On, chip works properly Low: Off, small current consumed
8	XPD_DCDC	I/O	Deep-sleep wakeup (need to be connected to EXT_RSTB); GPIO16
9	MTMS	I/O	GPIO 14; HSPI_CLK
10	MTDI	I/O	GPIO 12; HSPI_MISO
11	VDDPST	P	Digital/IO Power Supply (1.8 V ~ 3.6 V)
12	MTCK	I/O	GPIO 13; HSPI_MOSI; UART0_CTS
13	MTDO	I/O	GPIO 15; HSPI_CS; UART0_RTS
14	GPIO2	I/O	UART TX during flash programming; GPIO2
15	GPIO0	I/O	GPIO0; SPI_CS2
16	GPIO4	I/O	GPIO4
17	VDDPST	P	Digital/IO Power Supply (1.8 V ~ 3.6 V)
18	SDIO_DATA_2	I/O	Connect to SD_D2 (Series R: 20 Ω); SPIHD; HSPIHD; GPIO9
19	SDIO_DATA_3	I/O	Connect to SD_D3 (Series R: 200 Ω); SPIWP; HSPIWP; GPIO10
20	SDIO_CMD	I/O	Connect to SD_CMD (Series R: 200 Ω); SPI_CS0; GPIO11
21	SDIO_CLK	I/O	Connect to SD_CLK (Series R: 200 Ω); SPI_CLK; GPIO6
22	SDIO_DATA_0	I/O	Connect to SD_D0 (Series R: 200 Ω); SPI_MISO; GPIO7
23	SDIO_DATA_1	I/O	Connect to SD_D1 (Series R: 200 Ω); SPI_MOSI; GPIO8
24	GPIO5	I/O	GPIO5
25	U0RXD	I/O	UART Rx during flash programming; GPIO3
26	U0TXD	I/O	UART TX during flash programming; GPIO1; SPI_CS1
27	XTAL_OUT	I/O	Connect to crystal oscillator output, can be used to provide BT clock input
28	XTAL_IN	I/O	Connect to crystal oscillator input
29	VDDD	P	Analog Power 2.5 V ~ 3.6 V
30	VDDA	P	Analog Power 2.5 V ~ 3.6 V



Pin	Name	Type	Function
31	RES12K	I	Serial connection with a 12 k Ω resistor and connect to the ground
32	EXT_RSTB	I	External reset signal (Low voltage level: active)

Note:

1. *GPIO2, GPIO0, and MTDO are used to select booting mode and the SDIO mode;*
2. *U0TXD should not be pulled externally to a low logic level during the powering-up.*



3. Functional Description

The functional diagram of ESP8266EX is shown as in Figure 3-1.

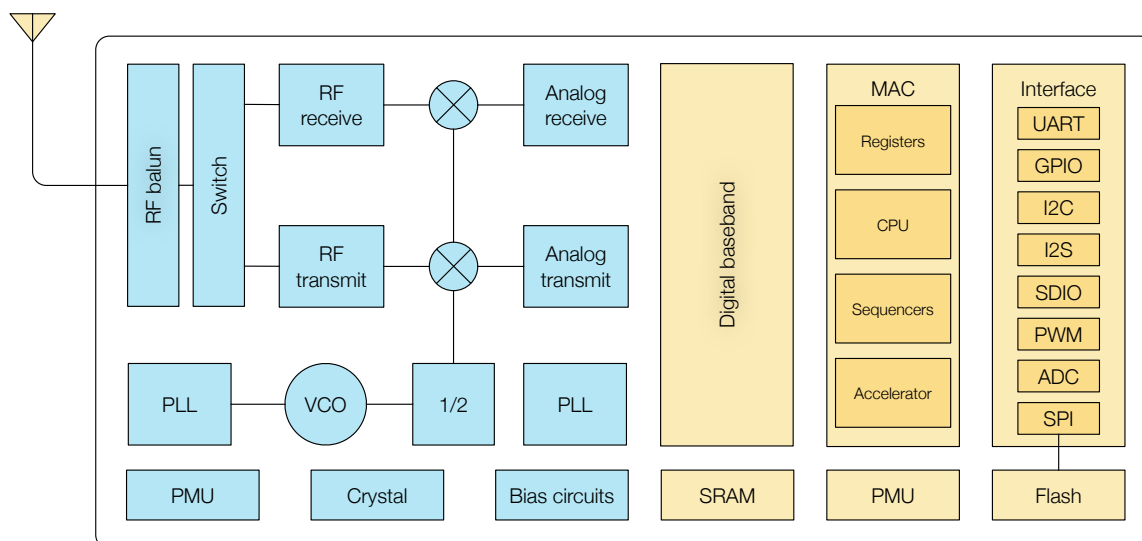


Figure 3-1. Functional Block Diagram

3.1. CPU, Memory, and Flash

3.1.1. CPU

The ESP8266EX integrates a Tensilica L106 32-bit RISC processor, which achieves extra-low power consumption and reaches a maximum clock speed of 160 MHz. The Real-Time Operating System (RTOS) and Wi-Fi stack allow 80% of the processing power to be available for user application programming and development. The CPU includes the interfaces as below:

- Programmable RAM/ROM interfaces (iBus), which can be connected with memory controller, and can also be used to visit flash.
- Data RAM interface (dBus), which can connected with memory controller.
- AHB interface which can be used to visit the register.

For information about the Xtensa® Instruction Set Architecture, please refer to [Xtensa® Instruction Set Architecture \(ISA\) Summary](#).

3.1.2. Memory

ESP8266EX Wi-Fi SoC integrates memory controller and memory units including SRAM and ROM. MCU can access the memory units through iBus, dBus, and AHB interfaces. All memory units can be accessed upon request, while a memory arbiter will decide the



running sequence according to the time when these requests are received by the processor.

According to our current version of SDK, SRAM space available to users is assigned as below.

- RAM size < 50 kB, that is, when ESP8266EX is working under the Station mode and connects to the router, the maximum programmable space accessible in Heap + Data section is around 50 kB.
- There is no programmable ROM in the SoC. Therefore, user program must be stored in an external SPI flash.

3.1.3. External Flash

ESP8266EX uses external SPI flash to store user programs, and supports up to 16 MB memory capacity theoretically.

The minimum flash memory of ESP8266EX is shown below:

- OTA disabled: 512 kB at least
- OTA enabled: 1 MB at least

! Notice:

SPI mode supported: Standard SPI, Dual SPI and Quad SPI. The correct SPI mode should be selected when flashing bin files to ESP8266. Otherwise, the downloaded firmware/program may not be working properly.

3.2. Clock

3.2.1. High Frequency Clock

The high frequency clock on ESP8266EX is used to drive both transmit and receive mixers. This clock is generated from internal crystal oscillator and external crystal. The crystal frequency ranges from 24 MHz to 52 MHz.

The internal calibration inside the crystal oscillator ensures that a wide range of crystals can be used, nevertheless the quality of the crystal is still a factor to consider to have reasonable phase noise and good Wi-Fi sensitivity. Refer to Table 3-1 to measure the frequency offset.

Table 3-1. High Frequency Clock Specifications

Parameter	Symbol	Min	Max	Unit
Frequency	FXO	24	52	MHz
Loading capacitance	CL	-	32	pF
Motional capacitance	CM	2	5	pF



Parameter	Symbol	Min	Max	Unit
Series resistance	RS	0	65	Ω
Frequency tolerance	Δ FXO	-15	15	ppm
Frequency vs temperature (-25 °C ~ 75 °C)	Δ FXO,Temp	-15	15	ppm

3.2.2. External Clock Requirements

An externally generated clock is available with the frequency ranging from 24 MHz to 52 MHz. The following characteristics are expected to achieve good performance of radio.

Table 3-2. External Clock Reference

Parameter	Symbol	Min	Max	Unit
Clock amplitude	VXO	0.8	1.5	Vpp
External clock accuracy	Δ FXO,EXT	-15	15	ppm
Phase noise @1-kHz offset, 40-MHz clock	-	-	-120	dBc/Hz
Phase noise @10-kHz offset, 40-MHz clock	-	-	-130	dBc/Hz
Phase noise @100-kHz offset, 40-MHz clock	-	-	-138	dBc/Hz

3.3. Radio

ESP8266EX radio consists of the following blocks.

- 2.4 GHz receiver
- 2.4 GHz transmitter
- High speed clock generators and crystal oscillator
- Bias and regulators
- Power management

3.3.1. Channel Frequencies

The RF transceiver supports the following channels according to IEEE802.11 b/g/n standards.

Table 3-3. Frequency Channel

Channel No.	Frequency (MHz)	Channel No.	Frequency (MHz)
1	2412	8	2447
2	2417	9	2452
3	2422	10	2457



Channel No.	Frequency (MHz)	Channel No.	Frequency (MHz)
4	2427	11	2462
5	2432	12	2467
6	2437	13	2472
7	2442	14	2484

3.3.2. 2.4 GHz Receiver

The 2.4 GHz receiver down-converts the RF signals to quadrature baseband signals and converts them to the digital domain with 2 high resolution high speed ADCs. To adapt to varying signal channel conditions, RF filters, automatic gain control (AGC), DC offset cancelation circuits and baseband filters are integrated within ESP8266EX.

3.3.3. 2.4 GHz Transmitter

The 2.4 GHz transmitter up-converts the quadrature baseband signals to 2.4 GHz, and drives the antenna with a high-power CMOS power amplifier. The function of digital calibration further improves the linearity of the power amplifier, enabling a state of art performance of delivering +19.5 dBm average TX power for 802.11b transmission and +18 dBm for 802.11n (MSC0) transmission.

Additional calibrations are integrated to offset any imperfections of the radio, such as:

- Carrier leakage
- I/Q phase matching
- Baseband nonlinearities

These built-in calibration functions reduce the product test time and make the test equipment unnecessary.

3.3.4. Clock Generator

The clock generator generates quadrature 2.4 GHz clock signals for the receiver and transmitter. All components of the clock generator are integrated on the chip, including all inductors, varactors, loop filters, linear voltage regulators and dividers.

The clock generator has built-in calibration and self test circuits. Quadrature clock phases and phase noise are optimized on-chip with patented calibration algorithms to ensure the best performance of the receiver and transmitter.

3.4. Wi-Fi

ESP8266EX implements TCP/IP and full 802.11 b/g/n WLAN MAC protocol. It supports Basic Service Set (BSS) STA and SoftAP operations under the Distributed Control Function



(DCF). Power management is handled with minimum host interaction to minimize active-duty period.

3.4.1. Wi-Fi Radio and Baseband

The ESP8266EX Wi-Fi Radio and Baseband support the following features:

- 802.11 b and 802.11 g
- 802.11 n MCS0-7 in 20 MHz bandwidth
- 802.11 n 0.4 μ s guard-interval
- up to 72.2 Mbps of data rate
- Receiving STBC 2 x 1
- Up to 20.5 dBm of transmitting power
- Adjustable transmitting power

3.4.2. Wi-Fi MAC

The ESP8266EX Wi-Fi MAC applies low-level protocol functions automatically, as follows:

- 2 $\times$ virtual Wi-Fi interfaces
- Infrastructure BSS Station mode/SoftAP mode/Promiscuous mode
- Request To Send (RTS), Clear To Send (CTS) and Immediate Block ACK
- Defragmentation
- CCMP (CBC-MAC, counter mode), TKIP (MIC, RC4), WEP (RC4) and CRC
- Automatic beacon monitoring (hardware TSF)
- Dual and single antenna Bluetooth co-existence support with optional simultaneous receive (Wi-Fi/Bluetooth) capability

3.5. Power Management

ESP8266EX is designed with advanced power management technologies and intended for mobile devices, wearable electronics and the Internet of Things applications.

The low-power architecture operates in the following modes:

- Active mode: The chip radio is powered on. The chip can receive, transmit, or listen.
- Modem-sleep mode: The CPU is operational. The Wi-Fi and radio are disabled.
- Light-sleep mode: The CPU and all peripherals are paused. Any wake-up events (MAC, host, RTC timer, or external interrupts) will wake up the chip.
- Deep-sleep mode: Only the RTC is operational and all other part of the chip are powered off.



Table 3-4. Power Consumption by Power Modes

Power Mode	Description	Power Consumption
Active (RF working)	Wi-Fi TX packet	Please refer to Table 5-2.
	Wi-Fi RX packet	
Modem-sleep ^①	CPU is working	15 mA
Light-sleep ^②	-	0.9 mA
Deep-sleep ^③	Only RTC is working	20 μ A
Shut down	-	0.5 μ A

Notes:

- ① **Modem-sleep** mode is used in the applications that require the CPU to be working, as in PWM or I2S applications. According to 802.11 standards (like U-APSD), it shuts down the Wi-Fi Modem circuit while maintaining a Wi-Fi connection with no data transmission to optimize power consumption. E.g., in DTIM3, maintaining a sleep of 300 ms with a wakeup of 3 ms cycle to receive AP's Beacon packages at interval requires about 15 mA current.
- ② During **Light-sleep** mode, the CPU may be suspended in applications like Wi-Fi switch. Without data transmission, the Wi-Fi Modem circuit can be turned off and CPU suspended to save power consumption according to the 802.11 standards (U-APSD). E.g. in DTIM3, maintaining a sleep of 300 ms with a wakeup of 3ms to receive AP's Beacon packages at interval requires about 0.9 mA current.
- ③ During **Deep-sleep** mode, Wi-Fi is turned off. For applications with long time lags between data transmission, e.g. a temperature sensor that detects the temperature every 100 s, sleeps for 300 s and wakes up to connect to the AP (taking about 0.3 ~ 1 s), the overall average current is less than 1 mA. The current of 20 μ A is acquired at the voltage of 2.5 V.



4. Peripheral Interface

4.1. General Purpose Input/Output Interface (GPIO)

ESP8266EX has 17 GPIO pins which can be assigned to various functions by programming the appropriate registers.

Each GPIO PAD can be configured with internal pull-up or pull-down (XPD_DCDC can only be configured with internal pull-down, other GPIO PAD can only be configured with internal pull-up), or set to high impedance. When configured as an input, the data are stored in software registers; the input can also be set to edge-trigger or level trigger CPU interrupts. In short, the IO pads are bi-directional, non-inverting and tristate, which includes input and output buffer with tristate control inputs.

These pins, when working as GPIOs, can be multiplexed with other functions such as I2C, I2S, UART, PWM, and IR Remote Control, etc.

4.2. Secure Digital Input/Output Interface (SDIO)

ESP8266EX has one Slave SDIO, the definitions of which are described as Table 4-1, which supports 25 MHz SDIO v1.1 and 50 MHz SDIO v2.0, and 1 bit/4 bit SD mode and SPI mode.

Table 4-1. Pin Definitions of SDIOs

Pin Name	Pin Num	IO	Function Name
SDIO_CLK	21	IO6	SDIO_CLK
SDIO_DATA0	22	IO7	SDIO_DATA0
SDIO_DATA1	23	IO8	SDIO_DATA1
SDIO_DATA_2	18	IO9	SDIO_DATA_2
SDIO_DATA_3	19	IO10	SDIO_DATA_3
SDIO_CMD	20	IO11	SDIO_CMD



4.3. Serial Peripheral Interface (SPI/HSPI)

ESP8266EX has two SPIs.

- One general Slave/Master SPI
- One general Slave/Master HSPI

Functions of all these pins can be implemented via hardware.

4.3.1. General SPI (Master/Slave)

Table 4-2. Pin Definitions of SPIs

Pin Name	Pin Num	IO	Function Name
SDIO_CLK	21	IO6	SPICLK
SDIO_DATA0	22	IO7	SPIQ/MISO
SDIO_DATA1	23	IO8	SPID/MOSI
SDIO_DATA_2	18	IO9	SPIHD
SDIO_DATA_3	19	IO10	SPIWP
U0TXD	26	IO1	SPICS1
GPIO0	15	IO0	SPICS2
SDIO_CMD	20	IO11	SPICS0

 Note:

SPI mode can be implemented via software programming. The clock frequency is 80 MHz at maximum when working as a master, 20 MHz at maximum when working as a slave.

4.3.2. HSPI (Master/Slave)

Table 4-3. Pin Definitions of HSPI

Pin Name	Pin Num	IO	Function Name
MTMS	9	IO14	HSPICLK
MTDI	10	IO12	HSPIQ/MISO
MTCK	12	IO13	HSPIID/MOSI
MTDO	13	IO15	HPSICS

 Note:

SPI mode can be implemented via software programming. The clock frequency is 20 MHz at maximum.



4.4. I2C Interface

ESP8266EX has one I2C, which is realized via software programming, used to connect with other microcontrollers and other peripheral equipments such as sensors. The pin definition of I2C is as below.

Table 4-4. Pin Definitions of I2C

Pin Name	Pin Num	IO	Function Name
MTMS	9	IO14	I2C_SCL
GPIO2	14	IO2	I2C_SDA

Both I2C Master and I2C Slave are supported. I2C interface functionality can be realized via software programming, and the clock frequency is 100 kHz at maximum.

4.5. I2S Interface

ESP8266EX has one I2S data input interface and one I2S data output interface, and supports the linked list DMA. I2S interfaces are mainly used in applications such as data collection, processing, and transmission of audio data, as well as the input and output of serial data. For example, LED lights (WS2812 series) are supported. The pin definition of I2S is shown in Table 4-5.

Table 4-5. Pin Definitions of I2S

I2S Data Input			
Pin Name	Pin Num	IO	Function Name
MTDI	10	IO12	I2SI_DATA
MTCK	12	IO13	I2SI_BCK
MTMS	9	IO14	I2SI_WS
MTDO	13	IO15	I2SO_BCK
U0RXD	25	IO3	I2SO_DATA
GPIO2	14	IO2	I2SO_WS

4.6. Universal Asynchronous Receiver Transmitter (UART)

ESP8266EX has two UART interfaces UART0 and UART1, the definitions are shown in Table 4-6.



Table 4-6. Pin Definitions of UART

Pin Type	Pin Name	Pin Num	IO	Function Name
UART0	U0RXD	25	IO3	U0RXD
	U0TXD	26	IO1	U0TXD
	MTDO	13	IO15	U0RTS
	MTCK	12	IO13	U0CTS
UART1	GPIO2	14	IO2	U1TXD
	SD_D1	23	IO8	U1RXD

Data transfers to/from UART interfaces can be implemented via hardware. The data transmission speed via UART interfaces reaches 115200 x 40 (4.5 Mbps).

UART0 can be used for communication. It supports flow control. Since UART1 features only data transmit signal (TX), it is usually used for printing log.

 Note:

By default, UART0 outputs some printed information when the device is powered on and booting up. The baud rate of the printed information is relevant to the frequency of the external crystal oscillator. If the frequency of the crystal oscillator is 40 MHz, then the baud rate for printing is 115200; if the frequency of the crystal oscillator is 26 MHz, then the baud rate for printing is 74880. If the printed information exerts any influence on the functionality of the device, it is suggested to block the printing during the power-on period by changing (U0TXD, U0RXD) to (MTDO, MTCK).

4.7. Pulse-Width Modulation (PWM)

ESP8266EX has four PWM output interfaces. They can be extended by users themselves. The pin definitions of the PWM interfaces are defined as below.

Table 4-7. Pin Definitions of PWM

Pin Name	Pin Num	IO	Function Name
MTDI	10	IO12	PWM0
MTDO	13	IO15	PWM1
MTMS	9	IO14	PWM2
GPIO4	16	IO4	PWM3

The functionality of PWM interfaces can be implemented via software programming. For example, in the LED smart light demo, the function of PWM is realized by interruption of the timer, the minimum resolution reaches as high as 44 ns. PWM frequency range is adjustable from 1000 μ s to 10000 μ s, i.e., between 100 Hz and 1 kHz. When the PWM



frequency is 1 kHz, the duty ratio will be 1/22727, and a resolution of over 14 bits will be achieved at 1 kHz refresh rate.

4.8. IR Remote Control

ESP8266EX currently supports one infrared remote control interface. For detailed pin definitions, please see Table 4-8 below.

Table 4-8. Pin Definitions of IR Remote Control

Pin Name	Pin Num	IO	Function Name
MTMS	9	IO14	IR TX
GPIO5	24	IO 5	IR Rx

The functionality of Infrared remote control interface can be implemented via software programming. NEC coding, modulation, and demodulation are supported by this interface. The frequency of modulated carrier signal is 38 kHz, while the duty ratio of the square wave is 1/3. The transmission range is around 1m which is determined by two factors: one is the maximum current drive output, the other is internal current-limiting resistance value in the infrared receiver. The larger the resistance value, the lower the current, so is the power, and vice versa.

4.9. ADC (Analog-to-Digital Converter)

ESP8266EX is embedded with a 10-bit precision SAR ADC. TOUT (Pin6) is defined as below:

Table 4-9. Pin Definition of ADC

Pin Name	Pin Num	Function Name
TOUT	6	ADC Interface

The following two measurements can be implemented using ADC (Pin6). However, they cannot be implemented at the same time.

- Measure the power supply voltage of VDD3P3 (Pin3 and Pin4).

Hardware Design	TOUT must be floating.
RF Initialization Parameter	The 107th byte of esp_init_data_default.bin (0 ~ 127 bytes), vdd33_const must be set to 0xFF.
RF Calibration Process	Optimize the RF circuit conditions based on the testing results of VDD3P3 (Pin3 and Pin4).
User Programming	Use system_get_vdd33 instead of system_adc_read .

- Measure the input voltage of TOUT (Pin6).



Hardware Design	The input voltage range is 0 to 1.0 V when TOUT is connected to external circuit.
RF Initialization Parameter	The value of the 107th byte of esp_init_data_default.bin (0 ~ 127 bytes), vdd33_const must be set to the real power supply voltage of Pin3 and Pin4. The unit and effective value range of vdd33_const is 0.1 V and 18 to 36, respectively, thus making the working power voltage range of ESP8266EX between 1.8 V and 3.6 V.
RF Calibration Process	Optimize the RF circuit conditions based on the value of vdd33_const . The permissible error is ± 0.2 V.
User Programming	Use <code>system_adc_read</code> instead of <code>system_get_vdd33</code> .

Notes:

esp_init_data_default.bin is provided in SDK package which contains RF initialization parameters (0 ~ 127 bytes). The name of the 107th byte in **esp_init_data_default.bin** is **vdd33_const**, which is defined as below:

- When **vdd33_const** = 0xff, the power voltage of Pin3 and Pin4 will be tested by the internal self-calibration process of ESP8266EX itself. RF circuit conditions should be optimized according to the testing results.
- When $18 \leq \text{vdd33_const} \leq 36$, ESP8266EX RF Calibration and optimization process is implemented via $(\text{vdd33_const}/10)$.
- When **vdd33_const** < 18 or $36 < \text{vdd33_const} < 255$, **vdd33_const** is invalid. ESP8266EX RF Calibration and optimization process is implemented via the default value 3.3 V.